

数字验证面霸 200 题

1. sv 的四值&二值逻辑类型

四值逻辑: integer、logic、reg、wire、tri

二值逻辑: byte、shortint、int、longint、bit

2. 如果没有赋值四值逻辑的返回值是什么(x), 二值逻辑的返回值是什么? (0)

3. 合并数组和非合并数组有什么区别?

合并数组是压缩, 非合并数组是非压缩

4. 合并数组和非合并数组的赋值方法?

合并数组可以进行整体赋值

5. 动态数组的使用方法步骤?

1. 声明 2. new[] 一开始需要分配空间/

复制方法: 也可以在调用 new[] 时将数组名也一并传递, 将已有数组的值复制到新数组中。Eg: dyn=new[20] (dyn);

6. 队列的优点?

(1) 【队列】结合了链表和数组的优点, 可以在它的任何地方添加或删除元素, 并且通过索引实现对任一元素的访问。

(2) 队列的声明是使用带有美元符号的下标:[\$], 队列元素的标号从 0 到\$. int q0[\$] = {0,1}; //队列创建并复制, 此时不再需要`

(3) 队列不需要 new [] 去创建空间, 你只需要使用队列的方法为其增减元素, 一开始其空间为 0。

(4) 队列的一个简单使用即是通过其自带方法 push_back () 和 pop_front () 的结合来实现 FIFO 的用法。(从 back 往里放, 从 front 往外拿)

7. 如何确定选择动态数组/队列/关联数组?

一、动态数组 最大特点: 在仿真运行时灵活调节数组的大小, 即存储量。

二、队列 具备链表和数组的优点, 可以在任何地方添加或删除元素, 并且通过索引访问任一元素。队列的声明使用: [\$]。队列元素的标号从 0 到\$。不需要使用 new[] 去创建空间, 只要使用队列的方法。队列的简单使用是通过自带的方法 push_back() 和 pop_front() 的结合来实现 FIFO 的用法。可以在一个队列中的任何地方增加或删除元素, 这类操作在性能上的损失比动态数组小得多

三、关联数组 用来保存稀疏矩阵的元素, 例如数组内标号不用按照顺序, 更加的灵活。需要对一个非常大的地址空间寻址时, 该数组只为实际写入的元素分配空间, 这种方法所需要的空间比定宽或动态数组所占空间要小的多

8. 动态数组的宽度怎么求?

\$size/.size

9. Function 和 task 的区别

区别: task 在使用时需要消耗时间, function 在使用时不会消耗时间, task 只能被 task 调用, function 可以被 task 和 function 调用, 因为 task 会消耗仿真时间所以 function 不能调用

10. Function 如果没有定义返回值类型, 有返回值吗? (有) 默认返回值类型是什么? (logic)

11. 如果不想有返回值应该怎么定义这个 function? Void 型

12. Break; continue; return 的含义, return 之后, function 里剩下的语句会执行吗

答: break 语句结束整个循环。continue 立即结束本次循环, 继续执行下一次循环。return 语句会终止函数的执行并返回函数的值(如果有返回值的话)。return 之后, function 里剩下的语句不能执行, 其是终止函数的执行, 并返回函数的值

13. modport 的作用

答: 可以指定方向(input/output)

14. 竞争和冒险现象都是什么?

竞争: 在组合逻辑电路中, 由于变量传输的路径不同, 传到输出端的时间不同, 竞争 冒险: 由于竞争引起的一些过度干扰脉冲(毛刺)

15. 如果使用 dve 工具该如何去观察这种竞争冒险现象?

delta cycle

16. 类的三大特性

封装、继承和多态

17. 分别介绍一下类的三大特性

封装: 采用封装的思想保证了类内部数据结构的完整性, 应用该类的用户不能轻易直接操纵该数据结构, 而只能执行该类允许公开的数据。这样可以避免外部对内部数据的影响, 提高程序的可维护性。

继承: 类的继承包括了继承父类的成员变量和成员方法。也就说一个子类在继承父类成员变量的同时也继承了父类操作这些成员变量的成员方法。

多态: 当一个类派生出子类的时候, 基类中的一些方法可能需要被重写。

18. 继承的关键词

extends

19. 子类可以指向父类吗? 父类可以指向子类吗?

父类可以指向子类, 但子类不可以指向父类

20. 如果要子类指向父类需要什么操作?

需要使用 Cast 向上类型转换/

21. 使用 cast 之前还有什么操作?

先把子类赋值给父类, 再使用 cast

22. SV 常用的通信手段 Mailbox 是怎么通信的?

①mailbox 是一中允许不同进程相互交换数据的方法, mailbox 是一个内置类, 本质上类似于队列, 但和 queue 队列的数据类型有很大不同, 使用 semaphore 来控制存储队列中的 push 和 pull。无法访问邮箱队列中的给定索引, 只能按照 fifo 的顺序检索项目。

②mailbox 前一个端口 put 进数据, 后面连接的端口 get 数据, 从而完成通信。

23. 随机化变量怎么写?

rand/randc

24. rand 和 randc 有什么区别?

答: rand 是根据位宽还有约束去随机前后两个可能会出现重复的就是相互之间的概率不影响, randc 是以轮询的方式随机的, 我这一轮随机不会出现重复的东西

25. 随机化约束怎么写?

constraint

26. 在约束中写权重分配的时候: =和:/的区别是什么?

答: =是单独每一个会出现的概率 ; /是全部会出现的概率

27. 如果两个约束冲突了，可能会出现什么问题？

可能会出错，写断言的时候也会出现错误的信息

28. 如果要解决约束冲突的问题，需要使用哪种方法？

软约束，soft

29. 如果写的代码中需要 sv 和 c 语言进行交互，一般会使用什么方法？

答：dpi/ import/export 等

30. 立即断言和并发断言的区别？

立即断言：立即断言检查当前仿真时间的条件，相当于 if else，需要放在过程块中。并发断言：可以在过程块、module、interface 和 program 块内定义并发断言 区别并发断言和立即断言的关键字是 property

31. 并发断言的用法的话，的三个层次分别是什么：

1. 序列 sequence 编写，将多个信号的关系用断言中特定的操作符进行表示；

2. 属性 property 的编写，它可以将多个 sequence 和多个 property 进行嵌套，外加上触发事件；

3. assert 的编写，调用 property 就可以。编写完断言后我们可以将它用在很多地方，比如 DUT 内部，或者在 top 层嵌入 DUT 中，还可以在 interface 处进行编写，基本能够检测到信号的地方都可以进行断言检测。

32. 在 class 中收集覆盖率和直接在 initial 中收集覆盖率有什么区别？

initial 中需要例化，class 不需要

33. 数据类型 logic、reg 和 wire 有什么区别？

Wire：用于连接不同的元件，不存储值，被连续赋值或者端口(port)驱动 Reg：并不意味实际的寄存器，代表 Verilog / SystemVerilog 中的数据存储元素，保存值，直到将下一个赋值(不是通过 assign 语句赋值)。可以被综合成触发器，锁存器或组合电路 Logic：不能被多驱动，改进了 Reg 数据类型，保存值，可以被连续赋值。

赋值的角度：(1)Logic：支持连续赋值、阻塞赋值和非阻塞赋值 (2)Reg：阻塞赋值和非阻塞赋值 (3)Wire：支持连续赋值

34. logic 类型可以代替 wire 类型和 reg 类型吗？

可以

35. 如果一个数据已经被 input/output 修饰过了还能用 logic 吗？inout 类型呢？

可以、不可以

36. sv 中有无符号数据类型

有符号：byte、shortint、int、longint、integer

无符号：bit、logic、reg、wire、tri

37. Sv 数据类型之间的转换

显式转换 1. 加单引号，静态转换，不会对转换值做检查，无法得知是否转换成功 2. \$cast

byte signed_value; //有符号

bit unsigned_value; //无符号

unsigned_value = unsigned'(signed_value) //将有符号类型数据，转换成无符号类型

隐式转换 1. 直接赋值转换

bit [4:0] unsigned_value0; //5位 无符号

bit [6:0] unsigned_value1; //7 位 无符号
unsigned_value1 = unsigned_value0; //此时会将 unsigned_value0 自动补齐到 7 位, 然后赋值给 unsigned_value1

38. 定义一个枚举类型按照%d 打印出来的结果是什么?

默认是 int 型从 0 开始

39. 枚举法的优点?

易读性强, 可维护性强

40. 如果想化简写法, 把函数用作用域符号的方式写在类外面需要什么方法?

extern

41. module 中的 function/task 默认是 automatic 型还是 static 型? class 中呢? 分别说明一下静态和动态的用法和区别?

static; automatic; 静态的会一直存在, 动态的在调用之后空间就会自动释放/
(注: 本身默认静态就不要关键词显式修饰, 本身动态就不用关键词显示修饰)

42. 接口的作用

1. 起到封装的作用, 把接口都写到 interface 中, 增加代码的可读性和可维护性

43. Clocking block 是干什么的?

在 interface 内部我们可以定义 clocking 块, 可以使得信号保持同步, 对于接口的采样和驱动有详细的设置操作, 从而避免 TB 与 DUT 的接口竞争, 减少我们由于信号竞争导致的错误。采样提前, 驱动落后, 保证信号不会出现竞争。

44. 子类重写父类的关键词

virtual

45. 使用虚方法的必要的条件?

必须是同一派系的类, 有继承关系

46. 深复制和浅复制的区别?

浅拷贝当第一层的属性值是基本数据类型时, 新的对象和原对象互不影响, 但是如果第一层的属性值是引用数据类型时, 那么新对象和原对象的属性值其指向的是同一块内存地址。

深拷贝会另外创建一个一模一样的对象, 新对象跟原对象不共享内存, 修改新对象不会改到原对象。

浅拷贝是将原始对象中的数据型字段拷贝到新对象中去, 将引用型字段的“引用”复制到新对象中去, 不把“引用的对象”复制进去, 所以原始对象和新对象引用同一对象, 新对象中的引用型字段发生变化会导致原始对象中的对应字段也发生变化。

深拷贝是在引用方面不同, 深拷贝就是创建一个新的和原始字段的内容相同的字段, 是两个一样大的数据段, 所以两者的引用是不同的, 之后的新对象中的引用型字段发生改变, 不会引起原始对象中的字段发生改变。

47. 线程和进程的关系?

包含关系: 一个进程至少有一个线程, 线程是进程的一部分, 所以线程也被称为轻权进程或者轻量级进程。

48. 多线程的同步调度方法

多线程之间同步主要由 mailbox、event、 semaphore 三种进行一个通信交互。

49. 什么时候使用 mailbox 调度?

主要用于两个线程之间的数据通信, 通过 put 函数和 get 函数还有 peek 函数进行数据的发送和获取。

50. 什么时候使用 event?

事件主要用于两个线程之间的一个同步运行, 通过事件触发和事件等待进行两个线程间的运行同步。使用@(event)或者 wait(event.trigger)进行等待, ->进行触发。

51. 什么时候使用 semaphore

旗语主要是用于对资源访问的一个交互, 通过 key 的获取和返回实现一个线程对资源的一个访问。使用 put 和 get 函数获取返回 key。一次可以多个。

52. 如何创建一个包? 包的引用方法?

```
package/endpackage; import pkg_name::*
```

53. package 中可不可以包含(或编译)module 和 interface?

不可以, package 中可以定义数据变量、方法(task/function)、方法声明、class、parameter、cover group 和 property(断言属性)

54. 约束的种类

布尔表达式 Boolean expressions

权重分配 Weighted distributions dist

范围表达式 Range expressions inside

条件表达式 Conditional expressions if else

外部约束

双向约束 SV 会同时计算所有的随机变量约束, 取它们的交集

55. 写出一个随机化的约束, 约束范围上下限?

```
c inside {[lo:hi]}; / !(c inside {[lo:hi]});
```

56. 规定在某一数组范围内随机化? \$指定上下限

```
rand bit [6:0] b; // 0 <= b <= 127
```

```
rand bit [5:0] e;
```

```
constraint c_range
```

```
{
```

```
    b inside {[$:4], [20:$]}; // 0 <= b <= 4 || 20 <= b <= 127
```

```
    e inside {[$:4], [20:$]}; // 0 <= e <= 4 || 20 <= e <= ?
```

```
}
```

使用数组

```
rand bit [6:0] b; // 0 <= b <= 127
```

```
int fib [5] = ' {1, 5, 3, 9, 2};
```

```
constraint c_range
```

```
{
```

```
    b inside fib;
```

```
}
```

57. 外部约束怎么写?

使用作用域符 constraint Packet::c_external {length == 1;} ...

58. 如何收集覆盖率?

主要分为三步: 1、定义; 2、例化, 建议采用 new(); 3、采样; 触发 covergroup 可以分为采样触发(sample)和事件触发(wait 或@)

59. byte 类型类型是双状态还是四状态?

双状态

60. byte 类型和 bit 类型可以指定位宽吗?

byte 不可以, bit 可以

61. 用多种方法定义一个大小 32bit 的数组

八行四列的定宽数组: `int arr[8][4];`/`int arr[0:7]:/bit [8:0]arr[4:0];`
等

62. 如何遍历一个二维数组?

`for/foreach`

63. 数组求和的函数? 使用 `sum` 时要注意什么问题?

`arr.sum/`; 要注意位宽截断问题, 是否需要强制类型转换

64. 动态数组 `new` 了 4 个空间之后给进行赋值, 再 `new[3]`; 打印出来的结果是什么?

全部为 0

65. 压缩结构体的关键词?

`packed`

66. 压缩结构体和非压缩结构体的区别?

一个地址连续一个不连续

67. 联合结构体的优点? 使用联合结构体时需要注意什么问题?

优点: 可以共享一个空间, 注意: 两个结构体所占用的空间大小必须一样大

68. 字符串类型是什么? 返回一个所有字符小写的字符串? 提取出 `started` 到 `end` 之间的所有字符?

`string/; Tolower(): /; S.Substr(started, end)`

69. `task` 和 `function` 的区别是什么?

区别: `task` 在使用时需要消耗时间, `function` 在使用时不会消耗时间, `task` 只能被 `task` 调用, `function` 可以被 `task` 和 `function` 调用, 因为 `task` 会消耗仿真时间所以 `function` 不能调用

70. 给 `task` 或者 `function` 传参时候, 如果参数缺省会发生什么问题?

如果参数列表中的变量给定过值, 就会使用参数列表中本身的值

如果参数列表中的变量, 只定义没有赋值, 就会报错

71. 从函数中如何返回一个数组?

1. `typedef (ppt)` 定义数组类型 (注: 如果在类中使用 `extern` 就不能自定义, 不然编译器识别不到自定义) 2. 使用 `ref` 3. `input/output`

72. 接口的作用 ?

起到封装的作用, 把接口都写到 `interface` 中, 增加代码的可读性和可维护性

73. `Clocking block` 的优点?

在 `interface` 内部我们可以定义 `clocking` 块, 可以使得信号保持同步, 对于接口的采样和驱动有详细的设置操作, 从而避免 TB 与 DUT 的接口竞争, 减少我们由于信号竞争导致的错误。采样提前, 驱动落后, 保证信号不会出现竞争。

74. `Wait` 和 `@` 的区别 ?

`@`: 如果 `clk` 一直没有变化 `@` 就无效, 不会触发下一条

`Wait`: `wait` 只需要 `clk` 为高, 就算一开始为高都触发

75. 接口可以在 `class` 中直接使用吗? 不能的话需要什么方法?

不能直接使用, 要用虚接口

76. 什么是 `oop`?

面向对象

77. 类中声明成员变量, 如果没有指明变量, 默认是什么类型? 需要写出来吗?

答: Public ; 不需要

78. 类中变量除了 public 还有什么类型?

答: protected (同一派系可以使用) /local (只能自己使用)

79. 作用是什么?

保护 ip 不修改内部变量

80. 句柄的传递, 比如说一个类 tran t1, t2; 声明两个句柄, 如果两个句柄 new 之后, t1=t2; 会产生什么效果?

t1 和 t2 指向同一个空间 (对象)。

81. fork_join_any 和 fork_join_none 的区别?

答: fork...join_any 对块内语句进行调度, 里边的任何一条语句完成后, 父线程继续执行。fork...join_none 在调度其块内语句时, 父线程继续执行。

82. 说一下 pre_randomize, post_randomize, randomize

①randomize() 函数对代码中的 rand 和 randc 属性的变量做随机初始化, randomize() 函数执行成功, 返回值为 1, 执行失败, 返回值为 0。

②pre_randomize() 和 post_randomize() 函数可以被用户重写, 用户使用 pre_randomize() 函数, 在随机前修改代码中非随机变量的值, 如上下限、权重等, post_randomize() 函数可以在随机后修改随机值或做一些其它的计算。

③执行顺序是 pre_randomize- randomize- post_randomize;

④randomize() 函数执行失败, 意味着代码中的约束 constrain 不可用, 代码中的随机变量将保持之前的值; randomize() 执行失败后, 不会执行后面的 post_randomize() 函数; randomize() 函数不能被重写 (override); randomize() 函数的随机种子可以被 srandom() 函数修改; randomize() 是 system verilog 的内置函数, 不是 VMM 和 UVM 的函数, 可以随时调用, 其只会修改代码中 rand 和 randc 修饰的变量的值。

83. 关闭某一个约束块使用什么方法?

答: .constraint_mode(0)

84. 关闭某一个随机变量用什么方法?

答: p.length.rand_mode(0);

85. 项目中会考虑哪些 coverage?

主要会考虑三个方面, 代码覆盖率, 功能覆盖率, 断言覆盖率。

代码覆盖率, 主要由行覆盖率、条件覆盖率、fsm 覆盖率、跳转覆盖率、分支覆盖率, 他们是否都是运行到的, 比如 fsm, 是否各个状态都运行到了, 然后不同状态之间的跳转是否也都运行到了。

功能覆盖率的话主要是自己编写 covergroup 和 coverpoint 去覆盖我们想要覆盖的数据和地址或者其他控制信号。

断言覆盖率主要检测我们的时序关系是否都运行到了, 比如总线的地址数据读写时序关系是否都有实现。

86. covergroup 和 coverpoint 是为了收集什么覆盖率?

功能覆盖率

87. 如果覆盖率的一直比较低, 会采取哪种方式?

1. 重新写一个随机种子 2. 修改随机化约束 3. 可以在 dve 中修改, 把覆盖率叠加到一起。

88. 声明和例化有什么区别 ?

声明是声明一个变量, 其中保存类对象的句柄

例化是通过构造函数，创建对象，分配内存空间，并将声明的句柄指向这段内存空间，虽然 sv 中可以在声明对象的同时例化对象，但是不建议如此，一般是在块语句之外声明类对象，在块语句内例化对象，可以控制对象的实例化顺序。

89. 什么是 UVM？它的优势是什么？

****UVM (Universal Verification Methodology) ****是一个标准化的用于验证设计的方法学。其优势包括：重用性、VIP 即插即用、通用性、独立于仿真器、支持 CDV (coverage driven verification)、支持 CRV (constraint random verification) 等等

90. uvm_component 和 uvm_object 有什么区别？

uvm_component：在 build_phase 之后就一直存在于整个仿真周期。通过 interface 连接到 DUT 或者通过 TLM port 连接到其他 uvm_component。通过 configure 机制和 phase 机制控制 uvm_component 的层次结构和仿真行为。

uvm_object：从一个 uvm_component 生成，然后传递到另一个 uvm_component 之后就会消失。不会连接到任何组件，也不存在 phase 机制

91. 为什么需要 phase 机制，不同的 phase 有什么区别？

phase 机制用来控制和同步不同 uvm_component 的仿真行为。

可以根据是否消耗仿真时间区分为 function phase 和 task phase

92. 哪些 phase 是 top-down phase、bottom-up phase 和 parallel phase？

build_phase 是 top-down phase，run_phase 等 task phase 是 parallel phase，其他都是 bottom-up phase。

93. 为什么 build_phase 是 top-down phase，而 connect_phase 是 bottom-up phase？

build_phase 需要验证平台根据高层次组件的配置来决定建立低层次的组件，所以其是 top-down phase。

connect_phase 需要在 build_phase 之后完成验证组件之间 TLM 连接

94. uvm phase 仿真是怎么开始启动的？

通过在顶层调用 run_test，run_test 任务会首先创建 test top，然后调用所有的 phase，开始仿真。

95. VCS 中通过什么方式执行对应的 test case？

在顶层调用 run_test()，在仿真选项中使用

+UVM_TESTNAME = testname

96. uvm_config_db 和 uvm_resource_db 有什么区别？

uvm_config_db 和 uvm_resource_db 都是参数化的类，用于配置验证组件中不同类型的参数。

相比于 uvm_resource_db，uvm_config_db 增加了层次关系。

对于最后配置的参数值，uvm_resource_db 是“last write wins”，uvm_config_db 是“parent wins”

97. 什么是 factory automation？

factory 是一种在验证环境中实例化组件或生成激励时提供更大灵活性的机制。

factory 机制允许在不修改原始源代码的条件下覆盖现有验证代码创建的对象类型。

98. 当前类的 super class 是什么？

super class 是当前类所**“扩展 (extends)”**的类。例如，my_test 的 super class 是 uvm_test

99. 什么是事物级建模 (Transaction Level Modelling) ?

事务级建模 (TLM) 是一种在较高抽象级别上对系统设计或模块设计进行建模的方法, 抽象出所有底层实现细节。

这是验证方法学中用于提高模块化和重用性的关键概念之一。即使 DUT 的实际接口由信号级别表示, 但大多数验证任务 (如生成激励, 功能检查, 收集覆盖率数据等) 都可以在事务级别上更好地完成。这有助于验证组件在项目内部和项目之间重用和维护。

100. 什么是 TLM ports 和 exports?

在事务级建模中, 不同的组件或模块使用事务对象进行通信。TLM port 定义了用于连接的一组方法 (API), 而这些方法的具体实现称为 TLM exports。

TLM port 和 export 之间的连接建立了两个组件之间的通信机制。producer 可以创建事务并将其 “put” 到 TLM 端口, 而 “put” 方法实现 (也称为 TLM export) 将在 consumer 中读取 producer 创建的事务, 从而建立通信渠道。

101. 什么是 TLM FIFOs?

如果 producer 组件和 consumer 组件需要独立运行, 则可以使用 TLM FIFO 进行事务通信。在这种情况下, producer 组件生成事务并将其 “puts” 到 FIFO, 而 consumer 组件一次从 FIFO 获取一个事务并对其进行处理。

102. TLM 上的 get () 和 peek () 操作之间有什么区别?

get () 操作将从 TLM FIFO 返回一个事务 (如果存在的话), 并且还会从 FIFO 中删除该事务。如果 FIFO 中没有可用的事务, 它将阻塞并等待, 直到 FIFO 至少有一个事务。

peek () 操作将从 TLM FIFO 返回事务 (如果存在的话), 而无需从 FIFO 中删除该事务。这也是一个阻塞调用, 如果 FIFO 没有可用事务, 它将等待。

103. TLM fifo 上的 get () 和 try_get () 操作之间有什么区别?

get () 是从 TLM FIFO 获取事务的阻塞调用。因此如果 FIFO 中没有可用的事务, 则任务 get () 将等待。

try_get () 是一个非阻塞调用, 即使 FIFO 中没有可用的事务, 也会立即返回。

try_get () 的返回值指示是否返回有效事务项。

104. analysisports 和 TLM ports 之间有什么区别? analysisFIFOs 和 TLM FIFO 有什么区别? analysis ports/ FIFO 在哪里使用?

TLM ports/ FIFO 用于两个组件之间的事务级别通信, 这两个组件具有使用 put/get 方法建立的通信通道。Analysis ports/FIFOs 是另一种事务通信通道, 用于组件将事务广播到多个组件。

TLM ports / FIFO 用于 driver 和 sequencer 之间的连接, 而 analysis ports/FIFOs 用于 monitor 将接收的事务广播到 scoreboard 或 reference model

105. sequence 和 sequence item 有什么区别?

sequence item 是一个对象, 其建模了两个验证组件之间传输的信息 (有时也可以称为事务 (transaction))。例如: 读操作和写操作中的地址和数据信息。

Sequence 是由 driver 驱动的 sequence item 序列模式, 由其 body () 方法实现。例如: 连续读取 10 次事务。

106. uvm_transaction 和 uvm_sequence_item 有什么区别?

uvm_transaction 是从 uvm_object 派生的用于对事务进行建模的基类。

sequence item 是在 uvm_transaction 的基础上还添加了一些其他信息的类, 例如: sequence id。建议使用 uvm_sequence_item 实现基于 sequence 的激励。

107. `copy()`、`clone()` 和 `create()` 方法之间有什么区别？

`create()` 方法用于构造一个对象。

`copy()` 方法用于将一个对象复制到另一个对象。

`clone()` 方法同时完成对象的创建和复制。

108. 解释 UVM 方法学中的 Agent 概念。

UVM agent 集合了围绕 DUT 的其他 uvm_components。大多数 DUT 具有多个 interfaces，并且一个 Agent 用于组合 driver、sequencer 和 monitor。使用此层次结构有助于在不同验证环境和项目之间重用 Agent。

109. `get_name()` 和 `get_full_name()` 有什么区别

`get_name()` 函数 返回对象的名称，该对象的名称由 new 构造函数或 `set_name()` 方法提供。

`get_full_name()` 返回对象的完整层次结构名称。对于 uvm_components，这在打印语句中使用时很有用，因为它显示了组件的完整层次结构。对于没有层次结构的 sequence 或配置对象，与 `get_name()` 打印相同的值

110. ACTIVE agent 与 PASSIVE agent 有何不同？

ACTIVE agent 是可以在其操作的接口上生成激励，其包含 driver 和 sequencer。PASSIVE agent 不生成激励，只能监视接口，这意味着在 PASSIVE agent 中将不会创建 driver 和 sequencer。

111. Agent 如何配置为“ACTIVE”或“PASSIVE”？

UVM agents 具有类型为 `uvm_active_passive_enum` 的变量，该变量定义 agents 是否是 active (UVM_ACTIVE)（构建了 sequencer 和 driver）或者 passive (UVM_PASSIVE)（不构建 sequencer 和 driver）。

此参数默认情况下设置为 UVM_ACTIVE，在 environment 类中创建 agents 时，可以使用 `**set_config_int()` 进行更改。然后，agent 的 build phase 阶段应具有以下代码以选择性地构建 driver 和 sequencer。

```
function void build_phase(uvm_phase phase);  
if(m_cfg.active == UVM_ACTIVE) begin  
//create driver, sequencer  
end  
endfunction
```

112. 什么是 driver 和 sequencer，为什么需要它们？

Driver 是根据接口协议将事务转换为一组信号级切换的组件。Sequencer 是一个将事务 (sequence items) 从 sequence 发送到 Driver，并将 Driver 的响应反馈给 sequence 的组件。

sequencer 也会对同时尝试访问 Driver 以激励设计接口的多个 sequences 进行仲裁。sequence 和 sequencer 在事务级抽象，Driver 在信号级对设计接口进行驱动，即单一设计模式。

113. UVM 中的 monitor 和 scoreboard 有什么区别？

monitor 是一个监测引脚级别的信号活动并将其转换为事务或 sequence_items 的组件。monitor 还通过 analysis port 将这些事务发送到其他组件。

scoreboard 用于检查 DUT 的行为是否正确。

114. 哪个方法可以激活 UVM 验证平台，如何调用它？

`**run_test()` 方法（静态方法）**用来激活 UVM 验证平台。通常在顶层的“initial begin...end”块中调用，并且它使用一个参数指定要运行的 test

case。run_test() 方法在 build_phase() 中执行 test class 的构造函数，并进一步构造层次化的 Env / Agent / Driver / Sequencer 对象。

115. 运行 sequence 需要什么步骤？

启动 sequence 需要执行三个步骤，如下所示：

创建一个序列。使用工厂创建方法创建一个序列：

```
my_sequence_c seq;
```

```
seq = my_sequence_c ::type_id :: create ( " my_seq" )
```

配置或随机化序列。

```
seq.randomize ()
```

开始一个 sequence。使用 sequence.start() 方法启动序列。start 方法需要输入一个指向 sequencer 的参数。关于 sequence 的启动，UVM 进一步做了封装。

116. 解释 sequencer 和 driver 之间的握手协议？

UVM 将 sequence item 从 sequence 传输到 driver 并收集来自 driver 的响应。

在 sequence 端：

```
**start_item() : **请求 sequencer 访问 driver
```

```
**finish_item() : **使 driver 接收 sequence item, 这是阻塞调用, 在 driver 调用 item_done() 方法后才返回。
```

在 driver 端：

```
**get_next_item(req):**这是 driver 中的一个阻塞方法, 直到接收到 sequence item, driver 可以将其转换为引脚级协议信号。
```

```
**item_done(req) : **向 sequencer 发出信号, 表明它可以接受新的 sequence 请求, 使得 sequence 解除对 finish_item() 方法的阻塞。
```

117. sequence 中的 pre_body() 和 post_body() 函数是什么？它们总是被调用么？

```
**pre_body() 是 sequence 类中的方法, 该方法在 body() 方法之前调用, 然后调用 post_body() **方法。
```

uvm_sequence::start() 有一个可选参数, 如果将其设置为 0, 将不调用这些方法。以下是 sequence 中 start() 方法的参数：

```
virtual task start (
```

```
uvm_sequencer_base sequencer, // Pointer to sequencer
```

```
uvm_sequence_base parent_sequencer = null, // parent sequencer
```

```
integer this_priority = 100, // Priority on the sequencer
```

```
bit call_pre_post = 1); // pre_body and post_body called
```

118. Sequencer 有哪些不同的仲裁机制可用于？

多个 sequences 可以同时连接到单个接口的 driver 上。Sequencer 支持仲裁机制，以确保在任何时间点只有一个 sequences 可以访问 driver。哪个 sequence_item 可以发送取决于用户选择的 Sequencer 仲裁算法。

在 UVM 中实现了五种内置的 Sequencer 仲裁机制，也可以实现用户自定义的仲裁算法。Sequencer 有一个 set_arbitration() 的方法，调用该方法可以选择使用哪种算法进行仲裁。可以选择的六种算法如下：

```
SEQ_ARB_FIFO(Default if none specified)
```

```
SEQ_ARB_WEIGHTED
```

```
SEQ_ARB_RANDOM
```

```
SEQ_ARB_STRICT_FIFO
```

SEQ_ARB_STRICT_RANDOM

SEQ_ARB_USER

119. 在 sequencer 上启动 sequence 时，如何指定 sequence 的优先级？

通过将优先级相对值参数传递给 sequence 的 start() 方法来指定优先级，第三个参数指定优先级。

```
seq_1.start(m_sequencer, this, 500); //Highest priority
```

```
seq_2.start(m_sequencer, this, 300); //Next Highest priority
```

```
seq_3.start(m_sequencer, this, 100); //Lowest priority among  
threesequences
```

120. sequence 如何才能独占访问 sequencer？

当在 sequencer 上运行多个 sequence 时，sequencer 对 sequence 中的每个 sequence item 的访问权限进行仲裁。有时一个 sequence 可能希望独占访问 sequencer，直到将其所有的 sequence item 都发送完毕为止（例如：你希望施加一个定向的激励，在此过程中不被打断）。

有两种机制允许 sequence 获得对 sequencer 的独占访问权。

lock() 和 unlock(): sequence 可以调用 sequencer 的 lock 方法，以通过 sequencer 的仲裁机制获得对 driver 的独占访问权限。如果还有其他标记为更高优先级的 sequences，则此 sequences 需要等待。授予 lock 权限后，其他 sequences 将无法访问 driver，直到该 sequences 在 sequencer 上调用 unlock() 方法。lock 方法是阻塞调用，直到获得 lock 权限才返回。

grab() 和 ungrab(): grab() 方法类似于 lock() 方法，用于申请独占访问。grab() 和 lock() 之间的区别在于，调用 grab() 时将立即生效，不考虑其他 sequences 的优先级，除非已经存在 sequences 调用了 lock() 或 grab()。

121. 流水线和非流水线 sequence-driver 模式有什么区别？

根据设计接口如何激励，可以分为两种 sequence-driver 模式：

非流水线模式：如果 driver 一次仅对驱动一个事务，则称为非流水线模型。在这种情况下，sequence 可以将一个事务发送给 driver，并且 driver 可能需要几个周期（基于接口协议）才能完成驱动该事务。只有在该事务完成驱动之后，driver 才会接受 sequencer 的新事务

```
class nonpipe_driver extends uvm_driver #(req_c);
```

```
task run_phase(uvm_phase phase);
```

```
req_c req;
```

```
forever begin
```

```
get_next_item(req); // Item from sequence via sequencer
```

```
// drive request to DUT which can take more clocks
```

```
// Sequence is blocked to send new items til then
```

```
item_done(); // ** Unblocks finish_item() in sequence
```

```
end
```

```
endtask: run_phase
```

```
endclass: nonpipe_driver
```

流水线模式：如果 driver 一次驱动多个事务，则称为流水线模式。在这种情况下，sequence 可以继续向 driver 持续发送新事务，而无需等待 driver 完成之前事务的驱动。在这种情况下，对于从该 sequence 发送的每个事务，driver 中都会有一个单独的进程来驱动该事务，而不用等到它完成上一次事务的驱动之后

再接受新事务。如果我们不需要等待设计的响应，则此模式很有用。

```
class pipeline_driver extends uvm_driver #(req_c);
task run_phase(uvm_phase phase);
    req_c req;
    forever begin
        get_next_item(req); // Item from sequence via sequencer
        fork
            begin
                //drive request toDUT which can take more clocks
                //separate threadthat doesn' t block sequence
                //driver can acceptmore items without waiting
            end
        join_none
        item_done(); //**Unblocks finish_item() in sequence
    end
endtask:run_phase
endclass:piepline_driver
```

122. 我们如何确保在 driver 驱动多个 sequences 时，则 driver 的响应会发送给正确的 sequences ？

如果从 driver 返回了几个 sequences 之一的响应，则 sequencer 利用 sequence 中的 sequence ID 字段将响应返回给正确的 sequence。driver 中的响应处理代码应调用 set_id_info()，以确保任何响应都具有与其原始请求相同的 sequence ID。下面的代码 driver 的参考伪代码说明上述问题。

```
class my_driver extends uvm_driver;
//function that gets item from sequence port and
//drives response back
function drive_and_send_response();
    forever begin
        seq_item_port.get(req_item);
        //function that takes req_item and drives pins
        drive_req(req_item);
        //create a new response item
        rsp_item = new();
        //some function that monitors response signals from dut
        rsp_item.data = m_vif.get_data();
        //copy id from req back to response
        rsp.set_id_info(req_item);
        //write response on rsp port
        rsp_port.write(rsp_item);
    end
endfunction
endclass
```

123. 什么是 m_sequencer 句柄？

启动 sequence 时，它始终与启动 sequencer 相关联。m_sequencer 句柄包含该

sequencer 的引用。使用此句柄，sequence 可以访问 UVM 组件层次结构中的信息

124. 什么是 p_sequencer 句柄，与 m_sequencer 相比有什么不同？

与 sequencer, driver 或 monitor 等 UVM 组件在整个仿真周期内都存在不同，UVM sequence 是生命周期有限的对象。因此，如果在 sequence 从测试平台层次结构（组件层次结构）访问任何成员或句柄，则需要运行该 sequence 的 sequencer 的句柄。

m_sequencer 是 uvm_sequencer_base 类型的句柄，默认情况下在 uvm_sequence 中可用。但是要访问在真实的 sequencer，我们需要对 m_sequencer 进行转换（typecast），通常称为 p_sequencer。下面是一个简单的示例，其中 sequence 要访问 clock_monitor 组件的句柄。

```
class test_sequence_c extends uvm_sequence;
test_sequencer_c p_sequencer;
clock_monitor_c my_clock_monitor;
task pre_body();
if(!$cast(p_sequencer, m_sequencer)) begin
`uvm_fatal("Sequencer Type Mismatch:", " Wrong Sequencer");
end
my_clock_monitor = p_sequencer.clk_monitor;
endtask
endclass
```

```
class test_Sequencer_c extends uvm_sequencer;
clock_monitor_c clk_monitor;
endclass
```

125. 生成 sequence 时，early randomization 和 late randomization 有什么区别？

在 early randomization 中，先调用 randomize() 方法对 sequence 对象进行随机化，然后再使用 start_item() 来请求对 sequencer 的访问，如下例所示：

```
task body()
assert(req.randomize());
start_item(req); //Can consume time based on sequencer arbitration
finish_item(req);
endtask
```

在 late randomization 中，先调用 start_item()，直到从 sequencer 授予仲裁，然后在将事务发送到 sequencer/driver 之前调用 randomize。

```
task body()
start_item(req); //Can consume time based on sequencer arbitration
assert(req.randomize());
finish_item(req);
endtask
```

126. 什么是 subsequence？

从 sequence 的 body() 任务中，如果调用了另一个 sequence 的 start()，则通常将其称为 subsequence。

127. get_next_item() 和 try_next_item() 有什么区别

`get_next_item()` 是一个阻塞调用，直到存在可供驱动的 `sequence item` 为止，并返回指向 `sequence item` 的指针。

`try_next_item()` 是非阻塞调用，如果没有可供驱动的 `sequence item`，则返回空指针。

128. UVM driver 类中的 `get_next_item()` 和 `get()` 方法之间有什么区别？

`get_next_item()` 是一个阻塞调用，用于从 sequencer FIFO 获取 `sequence item`。driver 驱动完 `sequence item` 后需要先使用 `item_done()` 完成握手，然后再使用 `get_next_item()` 请求新的 `sequence item`。

`get()` 也是一个阻塞调用，同样用于从 sequencer FIFO 获取 `sequence item`。但是在使用 `get()` 时，由于 `get()` 方法隐式完成了握手，因此无需显式调用 `item_done()`。

129. driver 中带和不带参数的 `item_done()` 调用有什么区别？

`item_done()` 方法是 driver 中的一种非阻塞方法，用于在 `get_next_item()` 或 `try_next_item()` 成功之后与 sequencer 完成握手。

如果不需要发回响应，则调用不带参数的 `item_done()`。

如果需要发回响应，则将指向 `sequence_item` 的指针作为 `item_done()` 参数。

130. 以下哪个 driver 类方法是阻塞调用哪些是非阻塞调用？

`get()`

`get_next_item()`

`item_done()`

`put()`

`try_next_item()`

`peek()`

`get()`, `get_next_item()`, `peek()` 是阻塞调用；

`try_next_item()`, `item_done()` 和 `put()` 非阻塞调用

131. 在 UVM driver 类中，以下哪个代码是错误的？

1) function `get_drive_req()`;

forever begin

req = `get()`;

req = `get()`;

end

endfunction

2) function `get_drive_req()`;

forever begin

req = `get_next_item()`;

req = `get_next_item()`;

`item_done()`;

end

endfunction

3) function `get_drive_req()`;

forever begin

req = `peek()`;

req = `peek()`;

`item_done()`;

```
req = get();  
end  
endfunction
```

2) 错误, 因为不能在调用 `item_done()` 之前两次调用 `get_next_item()`, 无法完成与 `sequencer` 的握手。

132. 如何停止在 `sequencer` 上运行的所有 `sequences`?

`sequencer` 具有 `stop_sequences()` 方法, 可用于停止所有 `sequences`。但是此方法不检查 `driver` 当前是否正在驱动任何 `sequence_items`, 如果 `driver` 调用 `item_done()` 或 `put()`, 则可能会出现 Fatal Error, 因为 `sequences` 指针可能无效。

133. 调用 `sequence.print()` 方法时, 应该调用 `sequence` 中的哪个方法?

`convert2string()`; 建议实现此函数, 该函数返回对象的字符串形式 (其数据成员的值), 这对于将调试信息打印到仿真器界面或日志文件很有用。

134. 找出 UVM `sequence` 的以下部分代码中的任何潜在问题

```
task body();  
seq_item_c req;  
start_item(req);  
#10 ns;  
assert(req.randomize());  
finish_item(req);  
endtask
```

应该避免在 `start_item` 和 `finish_item` 之间添加延迟。 `start_item` 返回后, 该 `sequence` 将赢得仲裁并可以访问 `sequencer/driver`。从那时起直到 `finish_item` 的任何延迟都将使得 `sequencer/driver` 暂停, 不能被其他 `sequence` 使用。

135. 什么是 `virtual sequence`, 在哪里使用? 有什么好处?

`virtual sequence` 是控制多个 `sequencers` 中激励生成的 `sequence`。由于 `sequences` `sequencers` 和 `drivers` 都只针对单个接口, 几乎所有测试平台都需要 `virtual sequence` 来协调不同接口之间的激励。

`Virtual sequences` 在子系统或系统级测试平台上也很有用, 可以使模块级的 `sequence` 协调地运行。

136. 使用 `new()` 和 `create()` 方法创建对象有什么区别?

UVM 推荐使用内置方法 `**::type id::create()`, 而不是直接调用构造函数 `new()` 创建组件或事务对象。

`create` 方法在内部调用 `factory` 机制以查找所请求创建的类型, 然后调用构造函数 `new()` 以实际创建一个对象而无需更改任何代码。

137. 我们如何注册 `uvm component` 类和 `uvm sequence` 类?

`uvm_sequence` 类使用 `uvm_object_utils()` 宏在 `factory` 中注册, 并将类名作为参数传递:

```
class test_seq_c extends uvm_sequence;  
`uvm_object_utils(test_seq_c)
```

`uvm_component` 类使用 `uvm_component_utils()` 宏在 `factory` 中注册, 并将类名作为参数传递:

```
class test_driver_c extends uvm_component;
```

```
`uvm_component_utils (test_driver_c)
```

138. 我们为什么要在 factory 中注册类?

factory 是 UVM 中使用的特殊查找表, 用于创建 component 或者 transaction 类型的对象。使用 factory 创建对象的好处是, 测试平台可以在运行仿真时决定创建哪种类型的对象。

一个类可以用另一个派生类替换, 而无需更改任何代码。为确保此功能, 建议所有类都在 factory 中注册。如果不向 factory 注册类, 则无法使用 factory 方法 `**::type id::create()` 构造对象。

139. instance override 和 type override 之间有什么区别?

顾名思义, type override 适用于该组件类型所在层次结构中的所有实例, instance override 仅覆盖类的特定实例, 即该组件在 UVM 组件层次结构中的位置。由于只有 UVM 组件类具有层次结构, 因此只有组件类能进行 instance override, 而 sequences (UVM objects) 只能进行 type overridden。

140. 什么是 objections 概念, 用在什么地方?

uvm_objection 类提供了一种在多个 components 和 sequences 之间共享计数器的方法。每个 component/sequence 可能会异步地“raise”和“drop”objections, 这会增加或减少计数值。当计数值达到零 (从非零值开始) 时, 则称发生“all dropped”条件。

objection 机制最常用于 UVM phase 机制中, 以协调每个 run-time phase 的结束。在用户进程开始时 raises an objection, 在用户进程结束时 drops the objection, 当一个 phase 中的所有用户进程都 drops objection 时, 该 phase 的 objections 计数将变为零。这种“all dropped”条件向 phase 机制表明, 每个用户进程都同意结束该 phase。

下面是一个示例说明如何在 sequencer (my-sequencer) 上启动 sequence (my testsequence), 并在 sequence 执行完成之后 drops objection。

```
task main_phase(uvm phase phase);  
phase. raise_objection(this );  
my_test sequence. start(my sequencer);  
phase. drop_objection(this);  
endtask
```

141. 我们如何在 UVM 中实现仿真超时机制 (simulation timeout mechanism)?

如果由于某些错误导致超出最大仿真时间, 仿真超时机制可以帮助停止仿真。在 UVM 中有一个很方便的函数 set_global_timeout(timeout) 用于将 uvm top.phase timeout 变量设置为超时值。如果 run () phase 在该超时值之前没有结束, 则仿真将停止并报告错误。

```
module test;  
initial begin  
set_global_timeout(100ons);  
end  
initial begin  
run test();  
end  
endmodule
```

142. UVM 方法学中 phase 的概念是什么？

不同于基于 module 的验证测试平台（所有 module 静态地存在于层次结构中），基于 class 的验证测试平台需要管理不同对象的创建和其方法的执行。

Phase 使得验证平台具有一致的执行流程。测试的执行可以分为以下任务

配置（configuration），

创建验证平台组件（creation），

运行激励（stimulus）和

测试检查结束（end of test checks）。

143. UVM 组件有哪些不同的 phases？UVM run-phase() 的 subphases 是什么？

UVM 使用标准 phases 来排序仿真过程中的主要步骤，有三组 phases，按以下顺序执行。

Build Phases—这些子 phases 都在 uvm_component 基类中作为 virtual 方法实现。

1) build_phase ()

2) connect_phase ()

3) end of elaboration () .

Run time phases—这些 phases 会消耗时间，这是大多数测试执行的地方。

1. start_of_simulation ()

2. run-phase ()，进一步分为 12 个子 phases:

1. pre_reset

2. reset

3. post reset

4. pre_configure

5. configure

6. post_configure

7. pre_main

8. main

9. post_main

10. pre_shutdown

11. shutdown

12. post_shutdown

Clean_up_phase—该清理阶段在测试结束后执行，用于收集并报告测试的结果和统计信息，包括以下子 phase:

1) extract ()

2) check ()

3) report ()

4) final ()

144. uvm——component 类中的 phase _ready_to_end() 方法有什么用途？

phase_ready_to_end(uvm_phase phase) 是 component 类中的回调 (callback) 方法，当该 phase 的所有 objections 均被 drop 之后调用该方法。component 类可以使用此回调方法来定义 phase 即将结束时需要执行的功能。

如果某个组件希望在 objections 被 drop 之后将 phase 结束延迟到某个条件，可以使用此回调方法来完成。

145. 什么是 uvm_config_db？它的作用是什么？

UVM configure 机制使用 uvm_config_db 配置数据支持在不同的测试平台组件之

间共享配置参数。任何测试平台组件都可以配置参数，其他组件可以从配置数据库访问这些参数，而无需知道其在层次结构中的位置。

例如，测试平台顶层可以通过 `uvm_config_db` 存储 virtual interface 句柄。然后任何 `uvm_driver` 或 `uvm_monitor` 组件都可以查询 `uvm_config_db` 以获取此 `virtual_interface` 的句柄，并将其用于获取实际接口信号。

146. 我们如何使用 `uvm_config_db` 的 `get()` 和 `set()` 方法？

`get()` 和 `set()` 是用于使用 `uvm_config_db` 配置或检索信息的主要方法。任何验证组件都可以使用 `set()` 方法为 `config_db` 配置一些信息，还可以控制其他组件对该信息的可见性。可以将其设置为全局可见性，或者仅对一个或多个特定测试平台组件可见。

`get()` 函数从数据库中匹配配置参数。`get()` 和 `set()` 方法的语法如下：

```
uvm_config_db#().set (uvm_componentcontext, string inst_name, string field_name, value)
```

```
uvm_config_db#().get(uvm_componentcontext, string inst_name, string field_name, ref value).
```

`context` 指定调用 `get/set` 的当前类或组件。

`inst_name` 是调用 `get/set` 的组件实例的名称。

`field_name` 是在 `config-db` 中 `get/set` 的参数的名称。

****标识 `get/set` 的配置信息的类型。

147. 验证平台中较低层次结构的组件是否可以使用 `get/setconfig` 方法将句柄传递给较高层次结构中的组件？

建议不要以这种方式在 UVM 中传递配置对象。通常，较高级别的组件设置配置数据而较低级别的组件使用 `get/set` 方法获取它们。

148. 在 UVM 方法学中仿真如何结束？

UVM 具有分阶段 (phase) 执行，该执行由一组构建阶段，运行阶段和检查阶段组成。

在运行阶段执行实际的仿真，并且在此阶段中每个组件都可以在开始时 `raise objection` 并保留该 `objection` 直到完成其仿真行为为止。一旦所有组件都 `drops objection` 则运行阶段完成，然后所有组件的 `check()` 阶段执行，最后测试结束。

这是正常仿真结束的方式，但是如果某些组件由于设计或测试平台中的错误而挂起，仿真超时 (timeouts) 也可以控制仿真结束。

149. 什么是 UVM RAL (UVM Register Abstraction Layer) ？

UVM RAL (寄存器抽象层) 是 UVM 支持的功能，有助于使用抽象寄存器模型来验证设计中的寄存器以及 DUT 的配置。UVM 寄存器模型寄存器抽象模型反映了寄存器设计的结构规范，提供了一种跟踪 DUT 寄存器内容和位置的方式。这是硬件和软件工程师的共同参考。RAL 的其他一些功能包括支持寄存器的 front door 和 backdoor 初始化以及内置的功能覆盖率支持。

150. 什么是 UVM Call back？

`uvm-callback` 类是用于实现回调 (callbacks) 的基类，这些回调用于在不更改组件类代码的情况下修改组件的行为。

一种常见用法是在 `driver` 将激励发送到 DUT 之前将错误注入到生成的数据包中。

以下伪代码显示了如何实现 Call back.

```
class Packet_c;
```

```

byte[4] src addr, dst addr;
byte[] data;
byte[4] crc;
endclass
//Userdefined callback class extended from base class
class PktDriver_Cb extends uvm_callback:
function new (string name = "pktDriver cb");
super. new(name);
endfunction
virtual task corrupt_packet (Packet_c pkt);
//Implement how to corrupt packet
//example -flip one bit of byte 0 in CRC
pkt.crc[0][0] = ~pkt. crc[0][0]
endtask
endclass: PktDriver_Cb
//Main Driver class
class PktDriver extends uvm_component;
uvm_component_utils(pktDriver) //Register callback class with
driveruvm_register_cb(PktDriver, PktDriver_Cb)
function new (string name, uvm_component parent=null);
super. new(name, parent);
endfunction
virtual task run();
forever begin
seq_item_port. get_next_item(pkt);
uvm do callbacks(PktDriver, PktDriver cb, corrupt_packet(pkt)) //other
code to derive to DUT etc end endtask endclass
1) 定义数据包 packet 类
2) 定义从 sequence 中接收此数据包并将其发送到 DUT 的 driver 类
3) 定义一个从 uvm_callback 基类派生的 driver callback 类，并添加一个虚拟方法，该方法
可用于注入错误或翻转数据包中的某个位。
4) 使用 uvm_register_cb() 宏注册 callback 类
5) 在 driver 类的 run() 方法中接收数据包并将其发送到 DUT
151. 什么是 uvm root 类?
uvm_root 类充当所有 UVM 组件的隐式顶层和 phase 控制器。用户不直接实例化
uvm_root，UVM 自动创建一个 uvm_root 实例，用户可以通过全局实例变量
uvm_top 访问。
152. uvm_test 的父类是什么?
uvm_test 类是用户可以实现的顶层类，并且没有提到显式父类。但是，UVM 有一个
称为 uvm_top 的特殊组件，它被指定为 test 类的父类。
153. 为异步 Fifo 编写 uvm_sequence_item
class seqItem extends uvm_sequence_item;
`uvm_object_utils(seqItem)
logic [8-1:0] rdata;
logic wfull;

```

```

logic rempty;
rand logic [8-1:0] wdata;
logic winc, wrst_n;
logic rinc, rrst_n;
function new(string name=" seqitem" );
    super.new(name);
endfunction
endclass

```

154. UVM 是否独立于 SystemVerilog?

UVM 建立在 SystemVerilog 基础上, 不支持 SystemVerilog 的工具也不会支持 UVM。

155. 什么是 virtual sequence 和 virtual sequencer?

virtual sequence 是一个包含和执行多个子 sequence 的容器, virtual sequencer 是包含其他 sequencer 的容器以使得 virtual sequence 中的每个子 sequence 都能在相应的 sequencer 上获得执行。

156. uvm_do 和 uvm_send 有什么区别?

uvm_do 自动地创建、随机化 和 发送 新的对象

uvm_send 用于发送已经完成创建和随机化之后的对象

157. 我们可以在 UVM 中自定义 phase 么?

可以在 UVM 中自定义 phase, 然后插入到已有的 phase 之间, 自定义 phase 继承自 uvm task phase, 需要实现其中的 exec task 和 exec func 方法。

158. RAL backdoor 和 frontdoor 访问有什么区别?

backdoor 访问: 通过 RTL 信号路径访问, 不消耗仿真时间

frontdoor 访问: 通过数据总线协议访问, 消耗仿真时间

159. set_config_* 和 uvm_config_db 有什么不同?

set_config_* 可以映射到相应的 uvm_config_db:

```
set_config_int(...) => uvm_config_db#(uvm_bit_stream_t) :: set(cntxt, ...)
```

```
set_config_string(...) > uvm_config_db#(string) :: set(cntxt, ...)
```

```
set_config_object (...) > uvm_config_db#(uvm_object) :: set(cntxt, ...)
```

160. 如何在 component 或者 sequence 中访问 DUT 中的信号?

接口信号可以通过指向具体 interface 的 virtual interface 访问, DUT 内部信号可以通过信号路径访问

161. 如何调试 UVM 中 config db 的名称或者路径不匹配的问题?

使用 +UVM_CONFIG_DB_TRACE 获得 set/get 信息

162. 如何连接 monitor 和 scoreboard, driver 和 sequencer?

使用 analysis_port 在 connect phase 连接 monitor 和 scoreboard, driver 具有一个 seq_item_port, 可以在 agent 的 connect phase 中连接到 sequencer 的 seq_item_export

163. 类中的静态属性和动态属性有什么区别?

SystemVerilog 支持属于**类 (class) 而不属于特定实例 (instance) ** 的静态属性。即使没有实例存在, 也可以访问这些属性。例如可以使用静态属性来计算特定类的实例数, 也可以调用可以访问静态属性的静态方法。静态方法无法访问非静态属性, 因为这些属性可能不存在。

```
virtual class car;
```

```
static int counter = 0; // shared by all instances
```

```

static local
function void increment counter();
counter ++
$display("creating item %0d counter);
endfunction
function new();
increment counter(); // increment on construction
endfunction: new
endclass: car

```

164. sequence 通常包含什么？

一个 sequence 通常包含一个 body 任务，在其中生成施加到 DUT 的激励。

165. 实现 AHB-Lite driver 的伪代码？

实现 AHB driver 的关键点是其流水线协议，需要在处理当前事务数据的同时处理下一事务的地址，这可以通过 fork-join 多个并行进程实现

```

class ahb_driver extends uvm_driver ;
semaphore sema ;
virtual task run_phase(uvm phasephase);
fork
drive_tx();
drive_tx();
join
endtask
virtual task drivetx();
//1. Gethold of a semaphore
//2. Gettransaction packet from sequencer
//3. Drivethe address phase
//4. Releasesemaphore
//5. Drive data phase
endtask
endclass

```

166. 什么是 UVM drain time？

drain time 是指设置的所有 objection 结束之后、仿真结束之前的时间

167. 仿真器需要设置 UVM HOME 才能仿真 uvm test 么？

当前主流仿真器都支持 UVM，可以不设置 UVM_HOME，这时候仿真工具会选择默认的 UVM 版本

168. 什么是 p-sequencer 句柄，与 m_sequencer 相比有什么不同？

与 sequencer, driver 或 monitor 等 UVM 组件在整个仿真周期内都存在不同，UVM sequence 是生命周期有限的对象。因此，如果在 sequence 从测试平台层次结构（组件层次结构）访问任何成员或句柄，则需要运行该 sequence 的 sequencer 的句柄。

m_sequencer 是 uvm_sequencer_base 类型的句柄，默认情况下在 uvm_sequence 中可用。但是要访问在真实的 sequencer，我们需要对 msequencer 进行转换（typecast），通常称为 p_sequencer。

下面是一个简单的示例，其中 sequence 要访问 clock monitor 组件的句柄。

```

class test_sequence_c extends uvm_sequence;
test_sequencer_c p_sequencer;
clock_monitor_c my_clock_monitor;
task pre_body();
if(!$cast(p_sequencer, m_sequencer)) begin
`uvm_fatal ("Sequencer Type Mismatch:", "wrong sequencer");
end
my_clock_monitor = p_sequencer. clk_monitor;
endtask
endclass
class test_sequencer_c extends uvm_sequencer;
clock_monitor_c clk_monitor;
endclass

```

169. UVM 验证环境的组成

driver: 向 sequencer 申请 sequence_item(数据包 transaction), 并将包里的信息按照总线协议规定驱动到 DUT 的端口上;

sequencer: 组织管理 sequence, driver 申请数据时, 它就把 sequence 生成的 sequence_item 发给 driver;

scoreboard: 比较 reference model 和 monitor 分别发送来的数据, 根据比较结果判断 DUT 是否正确工作;

monitor: 从 DUT 接收数据, 并将其转成 transaction 级别的 sequence_item, 发送给 scoreboard, 供其比较;

reference model: 模仿 DUT, 完成与 DUT 相同的功能, 为 scoreboard 提供判断标准;

agent: 将 sequencer、driver 和 monitor 封装在一起(UVM_ACTIVE/UVM_PASSIVE、两种模式), agent 模块的使用提高了代码的可重用性;

env: 将平台上的 component 组件封装在一起, 并配置各个组件间的通信端口, 实现一个环境多个用例。运行不同用例时, 在其中实例化 env 即可;

ase_test (uvm_test_top): 例化和配置共同的组件和 env, 其他的 test 继承 base_test, 并进行针对性的修改;

sequence: (不属于验证平台的任一部分) 产生激励内容 (transaction)。通过 sequence 中的 body 任务创建 (随机化) 事务, 并发送给 sequencer。

170. 在 UVM 方法学中仿真如何结束?

UVM 具有分阶段 (phase) 执行, 该执行由一组构建阶段, 运行阶段和检查阶段组成。

在运行阶段执行实际的仿真, 并且在此阶段中每个组件都可以在开始时 raise objection 并保留该 objection 直到完成其仿真行为为止。一旦所有组件都 drops objection 则运行阶段完成, 然后所有组件的 check () 阶段执行, 最后测试结束。

这是正常仿真结束的方式, 但是如果某些组件由于设计或测试平台中的错误而挂起, 仿真超时 (timeouts) 也可以控制仿真结束。

171. 如何在 UVM 中实现仿真超时机制

如果由于某些错误导致超出最大仿真时间, 仿真超时机制可以帮助停止仿真。在 UVM 中有一个很方便的函数 set global timeout(timeout) 用于将 uvm top. phase

timeout 变量设置为超时值。如果 run () phase 在该超时值之前没有结束，则仿真将停止并报告错误。

172. 工厂机制的好处是什么？

Factory 机制也叫工厂机制，其存在的意义就是为了能够方便的替换 TB 中的实例或者已注册的类型。一般而言，在搭建完 TB 后，我们如果需要对 TB 进行更改配置或者相关的类信息，我们可以通过使用 factory 机制进行覆盖，达到替换的效果，从而大大提高 TB 的可重用性和灵活性。

173. instance override 和 type override 之间有什么区别

顾名思义，type override 适用于该组件类型所在层次结构中的所有实例，instance override 仅覆盖类的特定实例，即该组件在 UVM 组件层次结构中的位置。由于只有 UVM 组件类具有层次结构，因此只有组件类能进行 instance override，而 sequences(UVM objects)只能进行 type overridden。

174. 通过工厂进行覆盖有什么要求？

无论是重载的类(parrot)还是被重载的类(bird)，都要在定义时注册到 factory 机制中。

被重载的类(bird)在实例化时，要使用 factory 机制的方式进行实例化，而不能使用传统的 new 的方式。

最重要的是，重载的类(parrot)要与被重载的类(bird)有派生关系。重载的类必须派生自被重载的类，被重载的类必须是重载类的父类。

component 与 object 之间互相不能重载。虽然 uvm_component 是派生自 uvm_object，但是这两者根本不能重载。因为，从两者的 new 参数的函数就可以看出来，二者互相重载时，多出来的一个 parent 参数会使 factory 机制无所适从。

175. 在 UVM 的工厂机制中，为什么要使用注册机制

只有先将类进行注册，在利用工厂方法创建之后才可以应用工厂机制提供的方法对已经实例化的类以及已经注册的类型进行覆盖。

176. 什么是 driver 和 sequencer，为什么需要它们

Driver 是根据接口协议将事务转换为一组信号级切换的组件。Sequencer 是一个将事务(sequence items)从 sequence 发送到 Driver，并将 Driver 的响应反馈给 sequence 的组件。

sequencer 也会对同时尝试访问 Driver 以激励设计接口的多个 sequences 进行仲裁。sequence 和 sequencer 在事务级抽象，Driver 在信号级对设计接口进行驱动，即单一设计模式。

177. monitor 执行的功能包括

观测 DUT 的 interface，并且收集总线信息。

永远保持 PASSIVE 模式，即永远不会驱动 DUT，只做激励，不做修改任何信号。

在总线协议或者内部信号协议观察时，可以做一些功能和时序的检查。

对于更加复杂的检查要求，它们可以将数据发送至其它验证组件，例如 scoreboard、reference model 或者 coverage collector。

178. scoreboard 执行的功能包括

scoreboard 进行数据比对和报告，在实际环境中，uvm_scoreboard 会接收来自多个 monitor 的监测数据，继而进行比对和报告。接收数据放到 buffer 里边，然后进行比对报告。

179. UVM 中如何在 driver 中使用 interface，为什么

Interface 如果不进行 virtual 声明的话是不能直接使用在 driver 中的, 会报错, 因为 interface 声明的是一个实际的物理接口。一般在 driver 中使用 virtual interface 进行申明接口, 然后通过 config_db 进行接口参数传递, 这样我们可以从上层组件获得 interface 实体进行处理。

180. Uvm_component_utils 有什么作用

factory 机制的实现被集成在了一个宏中: uvm_component_utils。这个宏最主要的任务是, 将字符串登记在 UVM 内部的一张表中, 这张表是 factory 功能实现的基础。只要在定义一个新的类时使用这个宏, 就相当于把这个类注册到了这张表中。这样, factory 机制可以实现: 根据一个字符串自动创建一个类的实例, 并且调用其中的函数 (function) 和任务 (task), 这个类的 main_phase 就会被自动调用。

181. 简单聊一下对 field_automation 机制的了解

该机制可以理解为: 对于 transaction extends uvm_sequence_item, 使用 uvm_field 宏注册的 transaction 属性, 可以使用一些 uvm 内置的方法。

182. 域的自动化有什么好处

可以自动实现 copy、compare、print 等三个函数。当使用 uvm_field 系列相关宏注册之后, 可以直接调用以上三个函数, 而无需自己定义。这极大的简化了验证平台的搭建, 尤其是简化了 driver 和 monitor, 提高了效率。

183. 简述 Config_db 的作用

config_db 机制主要作用就是传递参数使得 TB 的可配置性高, 更加灵活。Config_db 机制主要传递的有三种类型:

一种是 interface 虚拟接口, 通过传递 virtual interface 使得 driver 和 monitor 能够与 DUT 连接, 并驱动接口和采集接口信号。

第二种是单一变量参数, 如 int, string, enum 等, 这些主要就是为了配置某些循环次数, id 号是多少等等。

第三种是 object 类, 这种主要是当配置参数较多时, 我们可以将其封装成一个 object 类, 去包含这些属性和相关的处理方法, 这样传递起来就比较简单明朗, 不易出错。

184. 如果环境中有两个 config_db set, 哪个有效

UVM 更高的层次更接近用户, 为了让用户少和底层组件打交道, 所以层次越高优先级越高, 高层次的 set 会覆盖底层次的 set, 如果是层次相同再看时间先后顺序, 谁发生的晚谁有效, 时间靠后的会覆盖之前的。

185. UVM driver 类中的 get_next_item() 和 get() 方法之间有什么区别

get_next_item() 是一个阻塞调用, 用于从 sequencer FIFO 获取 sequence item。driver 驱动完 sequence item 后需要先使用 item_done() 完成握手, 然后再使用 get_next_item() 请求新的 sequence item。

get() 也是一个阻塞调用, 同样用于从 sequencer FIFO 获取 sequence item。但是在使用 get() 时, 由于 get() 方法隐式完成了握手, 因此无需显式调用 item_done()。

186. blocking 阻塞传输的方法包含哪几种

Put(): initiator 先生成数据 Tt, 同时将该数据传送到 target。

Get(): initiator 从 target 获取数据 Tt, 而 target 中的该数据 Tt 则应消耗。

Peek(): initiator 从 target 获取数据 Tt, 而 target 中的该数据 Tt 还应保留。

187. TLM 通信的步骤

- (1) 分辨出 initiator 和 target, producer 和 consumer。
- (2) 在 target 中实现 tlm 通信方法。
- (3) 在俩个对象中创建 tlm 端口。
- (4) 在更高层次中将俩个对象进行连接。

188. 说一下 peek 和 get 的差异

peek 是查看端口内部的数据事务但是不删除, get 是获取后立即删除。我们一般会先使用 peek 进行获取数据,但不删除(保证 put 端不会立马又发送一个数据),处理完毕后再用 get 删除。

189. Analysis port 是否可以不连或者连多个 import

都可以。Analysis port 类似于广播,其可以同时多个 imp 进行事务通信,只需要在每一个对应的 imp 端口申明 write() 函数即可。

对比 put.get, peek port, 他们都只能进行一对一传输,且也必须申明对应的函数如 put()、get()、peek()、can_put()/do_put() 等。

Fifo 是可以不用申明操作函数的,其内部封装了很多的通信端口,如 analysis_export 等,我们只需要将端口与其连接即可实现通信。

190. analysis port 的特点?

analysis port: 一端对多端

除了端对端的传输,在一些情况下还有多个组件会对同一个数据进行运算处理。

如果这个数据是从同一个源的 TLM 端口发出到达不同组件,这就要求该种端口可以满足从一端到多端的需求。

如果数据源端发生变化需要通知跟它关联的多个组件时,我们可以利用软件的设计模式之一观察者模式(observer pattern)来实现

191. get_name() 和 get_full_name() 有什么区别?

get_name() 函数返回对象的名称,该对象的名称由 new 构造函数或 set_name() 方法提供。

get_full_name() 返回对象的完整层次结构名称。对于 uvm_components,这在打印语句中使用很有用,因为它显示了组件的完整层次结构。对于没有层次结构的 sequence 或配置对象,与 get_name() 打印相同的值。

192. ACTIVE agent 与 PASSIVE agent 有何不同?

ACTIVE agent 是可以在其操作的接口上生成激励,其包含 driver 和 sequencer。PASSIVE agent 不生成激励,只能监视接口,这意味着在 PASSIVE agent 中不会创建 driver 和 sequencer。

193. 说一下 phase 机制的特点

在不同的时间做不同的事情,这是 phase 机制的特点,将 UVM 仿真阶段层次化,build phase 和 final phase 是自顶向下,其它都是自底向上,只有 run phase 是耗时的,为 task phase,其它都是 function phase

Run_phase 可以细分 12 个 phase, run_phase 与细分的 12 个 phase 是并行的,但是 12 个 phase 也是按先后顺序执行的。为了避免不必要的干扰,用户可以选择 run_phase,或者 12 个 phase 中的若干来完成激励,但是请不要将它们混合起来使用,因为这样容易导致执行关系的不明确。

所有的 phase 按照以下顺序自上而下自动执行:(九大 phase,其中 run phase 又分为 12 个小 phase)

phase	函数/任务	执行顺序	功能	典型应用
build	函数	自顶向下	创建和配置测试平台的结构	创建组件和寄存器模型, 设置或获取配置
connect	函数	自底向上	建立组件之间的连接	连接 TLM/TLM2 的端口, 连接寄存器模型和 adapter
end_of_elaboration	函数	自底向上	测试环境的微调	显示环境结构, 打开文件, 为组件添加额外配置
start_of_simulation	函数	自底向上	准备测试环境的仿真	显示环境结构, 设置断点, 设置初始运行的配置值
run	任务	自底向上	激励设计	提供激励, 采集数据和数据比较, 与 OVM 兼容
extract	函数	自底向上	从测试环境中收集数据	从测试平台提取剩余数据, 从设计观察最终状态
check	函数	自底向上	检查任何不期望的行为	检查不期望的数据
report	函数	自底向上	报告测试结果	报告测试结果, 将结果写入到文件中
final	函数	自顶向下	完成测试活动结束仿真	关闭文件, 结束联合仿真引擎

194. Phase 机制怎么同步?

Task phase 的同步, 一个 UVM 验证平台有许多 component 组成, 每个 component 都有自己的 run_phase, 以及从 pre_reset 到 post_shutdown 的 12 个小 phase。

只有所有 component 的每一个小 task phase 完成, 整个仿真平台才开始下一个小 task phase 的执行。各个 component 的 run_phase 之间, 以及 run_phase 于最后一个小 phase post_shutdown_phase 之间, 都有这样的同步。

195. function phase 和 task phase 区别

只有 run_phase (及其并行的 12 个 phase) 是 task phase (消耗仿真时间的 phase), 其他 phase 都是 function phase (不消耗仿真时间的非阻塞类型 phase)。

196. run_phase 和 main_phase 之间的关系

run_phase 和 main_phase (动态运行) 都是 task phase, 且是并行运行的, 后者称为动态运行 (run-time) 的 phase。如果想执行一些耗费时间的代码, 那么要在此 phase 下任意一个 component 中至少提起一次 objection, 这个结论只适用于 12 个 run-time 的 phase。对于 run_phase 则不适用, 由于 run_phase 与动态运行的 phase 是并行运行的, 如果 12 个动态运行的 phase 有 objection 被提起, 那么 run_phase 根本不需要 raise_objection 就可以自动执行。

197. 可以在 UVM 中自定义 phase 么

可以在 UVM 中自定义 phase, 然后插入到已有的 phase 之间, 自定义 phase 继承自 uvm task phase, 需要实现其中的 exec_task 和 exec_func 方法。

198. 为什么 build_phase 是 top-down phase, 而 connect_phase 是 bottom-up phase?

build_phase 需要验证平台根据高层次组件的配置来决定建立低层次的组件, 所以其是 top-down phase。

connect_phase 需要在 build_phase 之后完成验证组件之间 TLM 连接

199. UVM 层次化仿真通过什么机制进行调度的?

UVM 中运行是通过 Phase 机制进行层次化仿真的。从组件来看各个组件并行运行, 从 phase 上看是串行运行, 有层次化的。Phase 机制的 9 个 phase 是串行运行的, 不同组件中的同一个 phase 都运行完毕后才能进入下一个 phase 运行, 同一个 phase 在不同组件中的运行也是由一定顺序的, build 和 final 是自顶向下。

200. 如何理解 objection 机制?

UVM 中通过 objection 机制来控制验证平台的关闭, 需要在 drop_objection 之前先 raise_objection。验证在进入某一 phase 时, UVM 会收集此 phase 提出的所有 objection, 并且实时监测所有 objection 是否已经被撤销了, 当发现所有都已经撤销后, 那么就会关闭此 phase, 开始进入下一个 phase。当所有的

phase 都执行完毕后，就会调用\$finish 来将整个验证平台关掉。如果 UVM 发现此 phase 没有提起任何 objection，那么将会直接跳转到 下一个 phase 中。

UVM 的设计哲学就是全部由 sequence 来控制激励生成，因此一般情况下只在 sequence 中控制 objection。另外还需注意的是，raise_objection 语句必须在 main_phase 中第一个消耗仿真时间的语句之前。